

A Reproducible Support-Boundary Diagnostic for Tabular–Neural Reinforcement Learning under Exact-State Support Shift

Ercan Erkalkan^{a,*}

*^aVocational School of Technical Sciences, Department of Electronics and Automation,
Artificial Intelligence Operator Program, Marmara University, Mehmet Genç Campus,
34865 Kartal, Istanbul, Turkey*

Abstract

Hybrid reinforcement-learning controllers that combine exact-state memory with neural value functions must decide when memorized evidence is reliable and when neural extrapolation is being used outside observed support. This article presents a reproducible support-boundary diagnostic for tabular–neural Q-learning rather than a claim of universal algorithmic superiority. The study uses count gating, fixed mixing, support abstention, an approximate-support comparator, and a fuzzy support- and TD-residual-aware arbitration baseline. All experiments retain seed-level outcomes, exact environment-step checkpoints, read-only evaluation, isolated evaluation randomness, provenance records, and executable audits. In compact recurrent tasks, count gating improves over the validated DQN on FrozenLake 4x4, FrozenLake 8x8, and CliffWalking, but it does not generally outperform tabular Q-learning. Under complete held-out exact-state support in FourRooms, count and fuzzy gates degenerate to neural fallback and fail consistently, whereas support abstention repairs the specific unsupported preference mechanism. A 30-seed application-inspired navigation diagnostic with deployment goal shift shows the same trade-off: fuzzy arbitration improves AUC over the validated DQN by 0.575 (bootstrap 95% interval [0.152, 0.987]) but is worse than tabular Q-learning and support abstention, while abstention sharply reduces collisions by selecting a hold fallback at unsupported

*Corresponding author

Email address: `ercan.erkalkan@marmara.edu.tr` (Ercan Erkalkan)

URL: <https://orcid.org/0000-0001-9259-7112> (Ercan Erkalkan)

states. The main contribution is an auditable pre-deployment diagnostic: exact memory helps when states recur, adaptive arbitration helps only when its reliability signals are informative, and explicit fallback remains necessary at the support boundary.

Keywords: reinforcement learning, support-boundary diagnostic, fuzzy arbitration, deep Q-network, support shift, abstention, soft computing

1. Introduction

Tabular Q-learning stores an independently updated action value for each observed state [1]. This representation is transparent and rapidly correctable when exact states recur, but it has no mechanism for generalizing to an unseen state. Deep Q-networks (DQN) replace the table with a function approximator and use replay and target networks to stabilize learning [2, 3]. The substitution enables generalization while introducing optimization sensitivity, target error, and seed-dependent evaluation uncertainty [4, 5].

From a soft-computing perspective, this tension is central rather than incidental. Soft computing emphasizes approximate reasoning and adaptive computation under imprecision, uncertainty, and partial evidence [6]. A reinforcement-learning controller that acts in states outside its observed support is therefore not only optimizing expected return; it is also making a reliability-sensitive decision under incomplete evidence. The design question is how to arbitrate between a transparent memory estimate whose support is explicit and a neural estimate whose extrapolative reliability is not directly observed.

An intuitive soft-computing design combines both estimators. Episodic-control and memory-augmented deep-RL systems have demonstrated that fast non-parametric memory can improve learning [7, 8, 9, 10, 11]. Those methods use learned embeddings, nearest-neighbor retrieval, trajectories, or memory-based targets. Their success does not imply that a scalar mixture of an exact-state table and a neural value function is broadly effective. Such a mixture may only rescue a weak DQN when states recur, and may rely most strongly on unsupported neural extrapolation precisely when the table has no evidence.

This paper studies that boundary rather than proposing a generally superior algorithm. Compact environments are used as controlled diagnostic systems in which recurrence, exact-state support, transition noise, and held-

out goals can be varied without hiding the mechanism. The soft-computing motivation is reliability under partial support: counts provide transparent evidence, TD-residual statistics provide a lightweight uncertainty proxy, a fuzzy rule base converts these signals into a graded arbitration weight, and support abstention provides a conservative alternative when exact evidence is absent. This framing is relevant to the soft-computing community because hybrid intelligent systems must combine memorized evidence with learned predictors while remaining interpretable under uncertainty and distributional shift.

Real-life relevance arises in settings where reinforcement-learning components are considered for decision loops but exhaustive state coverage is unrealistic, such as mobile-robot navigation, unmanned-vehicle waypoint selection, adaptive scheduling, network routing, and operator-facing decision support. In these settings, a learned value function may be asked to act in rarely observed or shifted states, whereas an exact-memory branch is trustworthy only when its support is explicit. The present study does not claim real-world deployment; it provides a controlled pre-deployment diagnostic. If a hybrid agent fails when held-out support is known by construction, its behavior should not be assumed reliable under real distribution shifts. Support abstention is therefore interpreted as a safeguard for flagging unsupported states, deferring action, requesting additional evidence, or falling back to a verified controller rather than acting on unsupported neural extrapolation.

The contributions are deliberately limited to five auditable claims:

1. a reproducible support-boundary evaluation protocol for tabular-neural mixtures with exact interaction budgets, read-only evaluation, isolated random-number streams, raw seed retention, provenance, and executable audits;
2. confirmatory evidence that count gating can improve a development-selected and independently validated DQN only when exact states recur;
3. held-out-support evidence and proof of zero-support degeneration showing why count and fuzzy gates can fail when exact memory has no evidence;
4. an independently seeded confirmation-after-discovery replication showing that support abstention repairs that specific unsupported-neural-preference mechanism without becoming a general transfer method; and
5. a controlled application-inspired navigation diagnostic that reports re-

turn, success, collision, risk, unsupported-state exposure, branch use, abstention, exact-memory cost, and per-decision timing.

2. Related work

2.1. Episodic memory and tabular–neural combinations

Model-Free Episodic Control and Neural Episodic Control use non-parametric memory for rapidly updated values [7, 8]. Episodic Memory DQN adds memory values as learning targets [9]; Episodic Value Adjustment retrieves trajectories to adjust neural estimates [10]; and Generalizable Episodic Memory aggregates trajectories for implicit planning [11]. These methods are more expressive than the exact-state convex mixtures studied here. Our narrower design is useful because every source of behavior can be inspected and because a simple gate is easy to overinterpret.

Counts are often used for exploration. Pseudo-counts extend visitation bonuses to large observations [12], and Bootstrapped DQN uses approximate posterior sampling for deep exploration [13]. We instead use exact counts to arbitrate between estimators. A TD-residual gate is included as an inexpensive reliability heuristic, related in motivation but not statistical meaning to uncertainty based arbitration [14].

2.2. Fuzzy and uncertainty-aware arbitration

Fuzzy systems provide graded rules when evidence is incomplete rather than forcing a brittle binary boundary. Recent fuzzy deep-RL systems use prior knowledge, fuzzy representations, or adaptive rules to guide exploration and control [15, 16, 17]. These systems solve broader application problems than the exact-state mixture studied here. Their relevance is architectural: support and uncertainty can be mapped to an interpretable continuous arbitration signal instead of treated as a hard switch.

Uncertainty-aware navigation has likewise used predictive uncertainty to make collision-avoidance behavior more cautious in unfamiliar settings [18]. Our TD-residual proxy is intentionally lighter and is not a calibrated posterior. The fuzzy gate is therefore included as a soft-computing baseline, not as a claim that its output is a probability of correctness.

2.3. Support shift and abstention

Train–test separation reveals overfitting in deep RL [19], and generalization failures depend on the source and structure of distribution shift [20].

Offline-RL methods such as Batch-Constrained Q-learning and Conservative Q-learning limit poorly supported actions or values [21, 22]. Our online setting and neutral abstention rule are much simpler and are not claimed to be equivalent. The shared principle is that absence of support should not be confused with evidence that a learned extrapolation is reliable. Selective prediction formalizes a related risk–coverage trade-off by allowing a model to reject predictions [23]. Here, “abstention” concerns value-based action arbitration rather than supervised classification, and the application case maps rejection to a declared hold action. It is not a complete safety controller.

2.4. Reproducible reinforcement-learning evidence

RL rankings can change with seeds, implementation details, metrics, and aggregation [4, 5]. Reproducibility programs likewise emphasize executable artifacts and explicit evaluation assumptions [24]. We therefore retain all seeds, separate development from confirmation, report paired effects and robust sensitivity tests, preserve configuration hashes, and audit exact checkpoints and method coverage.

3. Methods

3.1. Estimators

For transition (s, a, r, s') , tabular Q-learning applies

$$\delta_{\text{tab}} = r + \gamma \max_{a'} Q_{\text{tab}}(s', a') - Q_{\text{tab}}(s, a), \quad (1)$$

$$Q_{\text{tab}}(s, a) \leftarrow Q_{\text{tab}}(s, a) + \alpha_{\text{tab}} \delta_{\text{tab}}. \quad (2)$$

The reported neural estimator is a two-hidden-layer MLP trained with replay, Huber loss, gradient clipping, and a target network. The development sweep compares vanilla DQN widths 64 and 128, learning rates 10^{-3} and 3×10^{-4} , replay capacities 50,000 and 100,000, target intervals 250–1,000, and a Double-DQN candidate. In Double DQN, the online network selects the next action and the target network evaluates it [3]. The statistical claims are intentionally restricted to this audited comparator family. The artifact also contains executable dueling Double-DQN and online advantage actor-critic (A2C) implementations, together with reviewer-response configuration files for full follow-up runs; those code paths are not used to claim stronger-neural-baseline performance in this article because the deposited results contain only smoke validation for them.

3.2. Mixtures and gates

The mixed action value is

$$Q_{\text{mix}}(s, a) = g(s)Q_{\text{tab}}(s, a) + [1 - g(s)]Q_{\text{net}}(s, a). \quad (3)$$

Fixed mixing uses $g(s) = 0.5$. Count gating uses

$$g_{\text{count}}(s) = \frac{N(s)}{N(s) + \tau}, \quad (4)$$

with exact-state visitation count $N(s)$ and development-selected $\tau = 20$. The TD-reliability gate uses exponentially weighted squared TD residuals, shrunk toward global estimates and clipped to $[0.05, 0.95]$. It is a heuristic arbitration signal, not calibrated uncertainty.

The fuzzy adaptive gate first computes

$$u_N(s) = 1 - \exp[-N(s)/\tau_N], \quad u_E(s) = \frac{\hat{E}_{\text{net}}(s)}{\hat{E}_{\text{net}}(s) + \kappa}, \quad (5)$$

where u_N is normalized exact-state support and u_E is a bounded neural uncertainty proxy obtained from the shrunk exponentially weighted squared TD residual. Triangular low, medium, and high support memberships and low/high uncertainty memberships activate five singleton rules. Their memory-weight consequents are 0, 0.35, 0.65, 0.75, and 0.95; weighted-average defuzzification yields $\alpha_{\text{fuzzy}}(s) \in [0.05, 0.95]$. At $N(s) = 0$, the memory weight is set explicitly to zero. The tested fuzzy baseline then uses the neural branch, allowing the experiment to determine whether soft arbitration can generalize across the support boundary. The logged quantities are α_{fuzzy} , u_N , u_E , and the selected branch.

At $N(s) = 0$, count gating sets $g_{\text{count}} = 0$ and therefore delegates completely to the neural estimator. The support-abstention rule instead uses

$$Q_{\text{abstain}}(s, a) = \begin{cases} 0, & N(s) = 0, \\ Q_{\text{mix}}(s, a), & N(s) > 0. \end{cases} \quad (6)$$

In the generic compact benchmarks, the zero vector invokes the same random tie-breaking as an uninitialized table. In the application environment, a declared fifth action holds position and is selected uniquely during abstention. Thus the application fallback is operationally distinct from a random move. This rule was discovered after the original support diagnostic and remains labeled “confirmatory replication after diagnostic discovery.”

Proposition 1 (Zero-support degeneration of count gating). *For any state s with exact-state count $N(s) = 0$, the count gate $g_{\text{count}}(s) = N(s)/(N(s) + \tau)$ with $\tau > 0$ reduces $Q_{\text{mix}}(s, a)$ to $Q_{\text{net}}(s, a)$ for every action a .*

Proof. Substituting $N(s) = 0$ gives $g_{\text{count}}(s) = 0/(0 + \tau) = 0$. Hence $Q_{\text{mix}}(s, a) = 0 \cdot Q_{\text{tab}}(s, a) + 1 \cdot Q_{\text{net}}(s, a) = Q_{\text{net}}(s, a)$. $\square$

Proposition 2 (Exposure bound of support abstention). *Let $\mathcal{U} = \{s : N(s) = 0\}$ be the set of unsupported exact states and let $\pi_{\text{net}}(s)$ denote the action selected from the neural branch. Under the support-abstention rule above, the number of decisions in which $\pi_{\text{net}}(s)$ is executed at states in $\mathcal{U}$ is zero, provided the declared fallback action is available.*

Proof. For every $s \in \mathcal{U}$, the policy evaluates the abstention branch rather than Q_{mix} . In the application case this branch assigns the unique maximal fallback value to the hold action, so no neural-branch preference can be selected at $s \in \mathcal{U}$. In compact benchmarks with no declared hold action, the zero vector still prevents a strict neural preference from determining the action; tie-breaking is explicitly logged as abstention rather than neural selection. $\square$

To test whether the exact-key definition is too brittle, the revised artifact also includes an approximate-support comparator. For a query vector $x(s)$ and previously observed vectors x_i , it computes

$$\widetilde{N}(s) = \sum_{i \in \mathcal{N}_k(s)} \exp \left[-\frac{\|x(s) - x_i\|_2^2}{2h^2} \right], \quad (7)$$

and uses the retrieved weighted tabular value

$$\widetilde{Q}_{\text{tab}}(s, a) = \frac{\sum_{i \in \mathcal{N}_k(s)} w_i Q_{\text{tab}}(s_i, a)}{\sum_{i \in \mathcal{N}_k(s)} w_i}. \quad (8)$$

The approximate-support gate is $g_{\text{approx}}(s) = \widetilde{N}(s)/(\widetilde{N}(s) + \tau_{\text{approx}})$. This baseline is intentionally separated from the reported exact-support claims: it evaluates whether nearby observed states can reduce the artificial hard boundary created by exact keys.

3.3. Observation encoding

Discrete Gymnasium observations are flattened to one-hot vectors for the network while retaining the integer as the exact table key. FourRooms uses normalized agent and goal coordinates (x, y, g_x, g_y) ; the complete vector is also the exact-state key. The application-navigation environment uses the same exact representation, so a held-out goal creates explicit support shift while remaining structured input to the DQN. Gymnasium 1.3 exposes Taxi as Taxi-v4, so a documented compatibility alias resolves the requested Taxi-v3 identifier to Taxi-v4 while recording both identifiers in result metadata. Every MiniGrid task uses `FullyObsWrapper` followed by `ImgObsWrapper`; incompatible partial and full observations are never mixed. MiniGrid and Gymnasium are described in [25, 26].

3.4. Evaluation controls and provenance

Training uses linearly decayed epsilon-greedy exploration. Evaluation is greedy with random tie-breaking and runs in a separate environment. Before an evaluation block, the agent random-number state is copied; after evaluation it is restored. Evaluation cannot update replay, either estimator, exact-state counts, or support-abstention diagnostics. Checkpoints occur at exact environment-step budgets.

Every raw row records the experiment and config, requested and resolved environment identifiers, observation representation, agent, seed, training steps, evaluation checkpoint and return, AUC inputs, wall-clock training and evaluation time, git commit, package version, Python, PyTorch, NumPy, Gymnasium, and MiniGrid versions. Evaluation rows additionally record failure, collision, risk-zone occupancy, unsupported-state ratio, memory and neural branch weights, abstention, fuzzy alpha and its within-episode interquartile range, support and uncertainty scores, exact-memory states/entries/estimated bytes, and mean/median action-decision time. The timer encloses action selection but excludes environment stepping. Resumable per-run shards are combined without discarding failed or unfavorable seeds.

4. Experimental design

4.1. Analysis families

Eight explicitly labeled evidence families are used:

1. **Development DQN tuning:** seeds 0–4 on seven compact tasks.
2. **Independent DQN validation:** seeds 600–629 on compact tasks and 700–709 on MiniGrid Empty-5x5.
3. **Confirmatory task expansion:** seeds 500–529 on seven compact tasks.
4. **Confirmation after diagnostic discovery:** seeds 300–329 on three held-out-goal FourRooms tasks and 400–429 on MiniGrid Empty-5x5.
5. **Extended MiniGrid diagnostic:** seeds 500–509 on six layouts.
6. **Application navigation case:** seeds 600–629 on the controlled deployment-goal shift.
7. **Adaptive-gate compact validation:** seeds 700–729 on FrozenLake 4x4, CliffWalking, and held-out-goal FourRooms 9.
8. **Cost and support analysis:** seeds 800–809 on FourRooms 9 and application navigation.

Development seeds choose the DQN configuration by lowest mean environment rank in return AUC, with worst rank, gradient updates, and agent name as fixed tie-breakers. No confirmatory, replication, or validation seed informs this choice. The released package also contains auxiliary dependency-free smoke validations for the approximate-support gate, fuzzy-parameter variants, and the dueling-DQN/A2C code paths. They are archived with the artifact as code-path checks only and are separated from the eight main evidence families in Table 1.

4.2. Tasks and budgets

The compact suite contains slippery FrozenLake 4x4 and 8x8, CliffWalking, Taxi, and held-out-goal FourRooms sizes 7, 9, and 11. FourRooms 9 and 11 use action-slip probability 0.1. Budgets range from 8,000 to 20,000 interactions. The MiniGrid diagnostic contains Empty 5x5, Empty 6x6, Empty 8x8, DoorKey 5x5, DoorKey 6x6, and FourRooms, with 20,000–40,000 interactions. The application case, implemented as `ApplicationNavigationSupportShift-v0`, is a 9×9 warehouse-style corridor layout with four staging starts, three recurring training goals, two shifted goals, four risk cells, collision penalties, 5% action slip, and a hold action. The state is $(x_{\text{agent}}, y_{\text{agent}}, x_{\text{goal}}, y_{\text{goal}})$ normalized to $[0, 1]^4$. The five actions are north, east, south, west, and hold. A successful goal arrival gives reward +5.0 and terminates the episode; each non-terminal step gives -0.02 ; a collision with a wall or map boundary adds -0.23 ; and occupying a risk

Table 1: Eight main executed ASOC result families. All listed audits passed; auxiliary smoke checks are archived in the artifact but not used as performance evidence.

Result family	Status	Environments	Methods	Runs	Analysis
DQN tuning development	PASS	7	7	245	development model selection
DQN strong validation	PASS	8	2	440	independent validation after development selection
Confirmatory extended compact	PASS	7	7	1470	confirmatory task expansion
Support-abstention replication	PASS	4	7	840	confirmatory replication after diagnostic discovery
MiniGrid extended diagnostic	PASS	6	7	420	post hoc extended MiniGrid diagnostic
Application navigation case study	PASS	1	6	180	confirmatory application case study
Adaptive-gate compact validation	PASS	3	5	450	confirmatory adaptive-gate validation
Cost-support metrics	PASS	2	5	100	descriptive cost and support analysis

cell adds -0.08 . Training uses only recurring goals; deployment evaluation samples both recurring and shifted goals. Each of 30 seeds receives 12,000 training interactions and six evaluation checkpoints with 30 episodes each. This is an application-inspired controlled pre-deployment diagnostic for mobile-agent, unmanned-vehicle, or operator-facing routing decisions, not a physical deployment. Table 1 reports execution coverage.

4.3. Outcomes and statistics

The primary metric is trapezoidal evaluation-return area under the measured learning curve (AUC). Secondary metrics include final return, success-rate AUC, runtime, gradient updates, gate behavior, visited states, collision and risk, unsupported-state ratio, branch usage, abstention, fuzzy alpha, exact-memory cost, and action-decision time. The estimated exact-memory footprint is the number of float32 state-action values plus one int64 count per stored state; it excludes Python-container and neural-network storage and is therefore a transparent lower-bound estimate. Paired comparisons report mean and median differences, percentile bootstrap 95% intervals, paired t tests, Wilcoxon signed-rank sensitivity tests, win/loss/tie counts, Cohen’s d_z , and paired rank-biserial correlation. Holm correction is applied across each configured planned-contrast family. Heavy-tail diagnostics report skew, excess kurtosis, and robust catastrophic outlier counts. Cross-environment summaries use ranks rather than pooling incommensurate raw returns.

5. Results

5.1. RQ1: Does count gating help over a validated DQN when states recur?

Figure 1 and Table 2 show the expanded compact family. Count gating improves AUC over the validated DQN on FrozenLake 4x4 by 0.316 (bootstrap 95% interval $[0.268, 0.365]$, t -Holm $p = 7.78 \times 10^{-12}$), FrozenLake 8x8 by

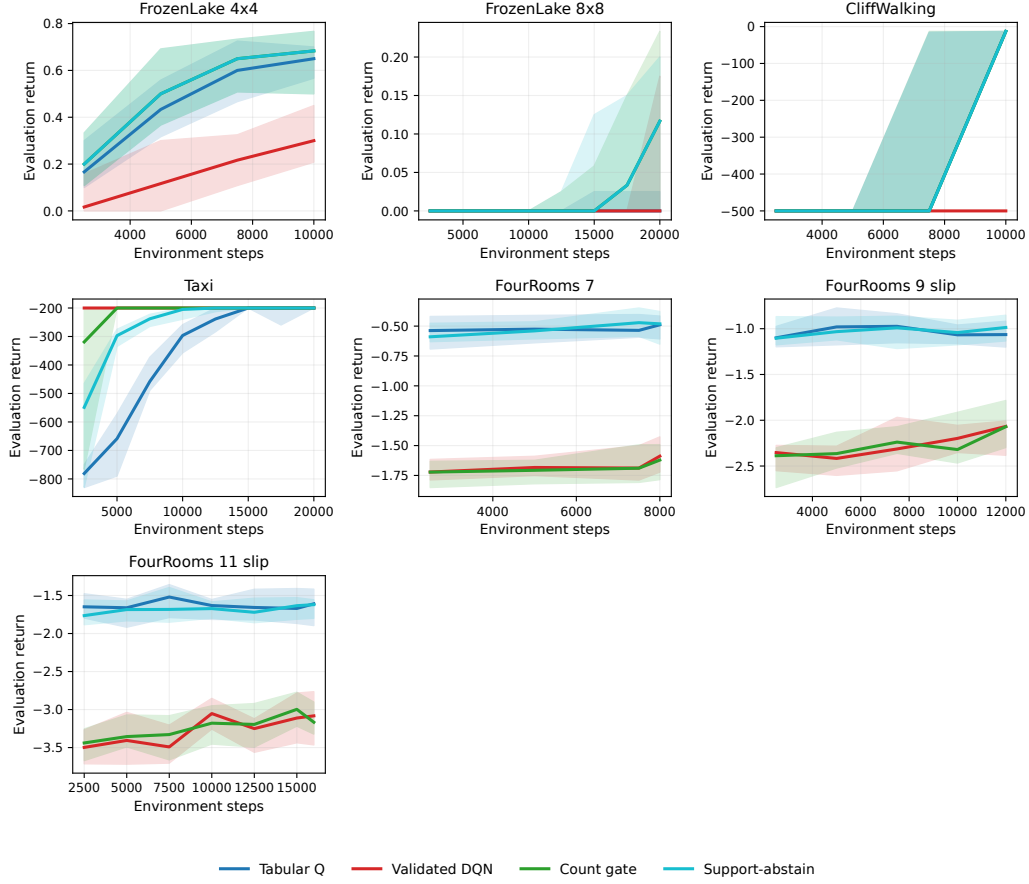


Figure 1: Median evaluation-return learning curves and interquartile bands for the seven compact tasks. Curves use 30 independently seeded runs per method.

0.027 ($[0.014, 0.041]$, $p = 0.0126$), and CliffWalking by 143.3 ($[113.6, 173.1]$, $p = 6.53 \times 10^{-9}$). These are the support-dependent positive results. The count gate does not significantly improve over the validated DQN on any held-out-goal FourRooms task and is worse in mean AUC on Taxi (-11.9 , interval $[-23.9, -0.3]$), although that Taxi contrast does not survive Holm correction.

5.2. RQ2: Does count gating outperform tabular Q-learning?

No general advantage is supported. Count gating and tabular Q-learning are not significantly different on CliffWalking or FrozenLake 4x4 after correction. Count gating improves over tabular learning on Taxi by 154.4 ($[135.8,$

Table 2: Return AUC for core methods in the confirmatory compact expansion. Intervals are seed-bootstrap 95% intervals.

Environment	Method	Seeds	Mean AUC [bootstrap 95% CI]	Median AUC
CliffWalking	Count gate	30	-356.672 [-386.389, -326.911]	-419
CliffWalking	Support-abstain	30	-381.589 [-442.400, -337.811]	-419
CliffWalking	Tabular Q	30	-386.363 [-438.387, -340.800]	-419
CliffWalking	Validated DQN	30	-500.000 [-500.000, -500.000]	-500
FourRooms 11 slip	Count gate	30	-3.233 [-3.295, -3.175]	-3.22
FourRooms 11 slip	Support-abstain	30	-1.671 [-1.701, -1.642]	-1.68
FourRooms 11 slip	Tabular Q	30	-1.649 [-1.678, -1.618]	-1.65
FourRooms 11 slip	Validated DQN	30	-3.297 [-3.344, -3.250]	-3.32
FourRooms 7	Count gate	30	-1.687 [-1.722, -1.652]	-1.69
FourRooms 7	Support-abstain	30	-0.522 [-0.548, -0.495]	-0.527
FourRooms 7	Tabular Q	30	-0.538 [-0.569, -0.507]	-0.537
FourRooms 7	Validated DQN	30	-1.667 [-1.701, -1.633]	-1.67
FourRooms 9 slip	Count gate	30	-2.265 [-2.319, -2.210]	-2.25
FourRooms 9 slip	Support-abstain	30	-1.022 [-1.053, -0.991]	-1.03
FourRooms 9 slip	Tabular Q	30	-1.034 [-1.072, -0.998]	-1.02
FourRooms 9 slip	Validated DQN	30	-2.303 [-2.363, -2.241]	-2.32
FrozenLake 4x4	Count gate	30	0.525 [0.486, 0.562]	0.556
FrozenLake 4x4	Support-abstain	30	0.525 [0.486, 0.562]	0.556
FrozenLake 4x4	Tabular Q	30	0.494 [0.461, 0.529]	0.481
FrozenLake 4x4	Validated DQN	30	0.209 [0.175, 0.244]	0.189
FrozenLake 8x8	Count gate	30	0.042 [0.026, 0.060]	0.0202
FrozenLake 8x8	Support-abstain	30	0.047 [0.029, 0.068]	0.0298
FrozenLake 8x8	Tabular Q	30	0.010 [0.004, 0.019]	0.00238
FrozenLake 8x8	Validated DQN	30	0.015 [0.009, 0.022]	0.00595
Taxi	Count gate	30	-224.226 [-236.632, -213.394]	-209
Taxi	Support-abstain	30	-259.430 [-266.809, -252.208]	-257
Taxi	Tabular Q	30	-378.605 [-392.533, -365.890]	-366
Taxi	Validated DQN	30	-212.341 [-225.605, -201.426]	-200

173.0], t -Holm $p = 1.62 \times 10^{-14}$), but is substantially worse on all three held-out-goal FourRooms tasks: mean deficits are 1.149, 1.231, and 1.584, with all 30 pairs favoring tabular learning in each task. The FrozenLake 8x8 mean gain of 0.032 does not survive the mean-based Holm family ($p = 0.0947$). Across the seven compact tasks, support abstention has the best mean environment rank (2.07), count gating ranks second (2.93), and tabular learning ranks third (3.43); these ranks are descriptive and do not imply a common-scale performance effect.

5.3. RQ3: What happens under held-out exact-state support shift?

In FourRooms, changing the goal changes the exact state key while preserving a structured coordinate input for the network. Count gating, fixed mixing, and the validated DQN all obtain strongly negative AUCs. Tabular and random tie-breaking are much less negative. This pattern is consistent

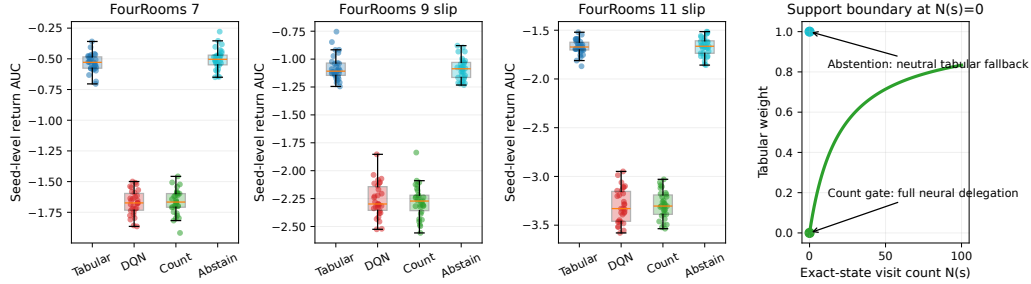


Figure 2: Support boundary and neural extrapolation failure. The first three panels show the independently seeded replication. The final panel shows why the standard count gate delegates completely to the neural estimator at $N(s) = 0$, whereas support abstention uses a neutral fallback.

Table 3: Support-abstention versus count-gating replication after diagnostic discovery. Positive differences favor abstention.

Environment	Pairs	Mean difference [bootstrap 95% CI]	t-Holm p	Wilcoxon-Holm p	Wins/losses/ties
FourRooms 11 slip held-out	30	1.625 [1.560, 1.689]	4.54e-28	3.73e-08	30/0/0
FourRooms 7 held-out	30	1.164 [1.117, 1.211]	6.06e-28	3.73e-08	30/0/0
FourRooms 9 slip held-out	30	1.190 [1.142, 1.238]	7.98e-28	3.73e-08	30/0/0
MiniGrid Empty 5x5	30	0.019 [-0.034, 0.070]	1	1	7/4/19

across sizes 7, 9, and 11: low or zero support increases reliance on neural extrapolation, but that extrapolation is not reliable. Figure 2 links the empirical result to the gate definition.

5.4. RQ4: Does support abstention replicate the specific repair?

Yes for the discovered FourRooms mechanism, not universally. Table 3 uses new seeds 300–329. Support abstention improves over count gating by 1.164 on FourRooms 7 ([1.117, 1.211]), 1.190 on FourRooms 9 ([1.142, 1.238]), and 1.625 on FourRooms 11 ([1.560, 1.689]). All 30 pairs favor abstention in every environment; mean-based Holm values are below 8.0×10^{-28} and Wilcoxon-Holm values are 3.73×10^{-8} . Support abstention is not superior to tabular learning after correction on these tasks, which indicates repair of harmful extrapolation rather than new transfer.

On the independent MiniGrid Empty-5x5 replication, abstention and count gating are not different (mean difference 0.019, interval $[-0.034, 0.070]$, t -Holm $p = 1$). This neutral replication is important: support abstention is a boundary-specific repair, not a universal improvement.

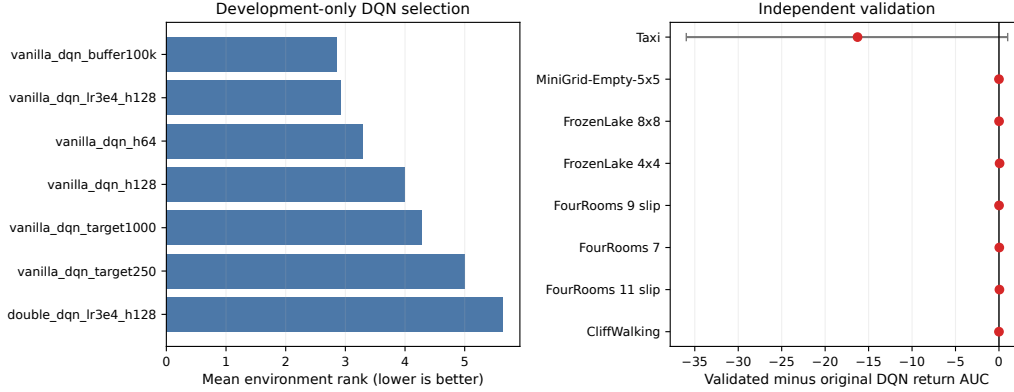


Figure 3: DQN development selection and independent validation. Lower development rank is better. Validation differences are selected minus original DQN AUC with bootstrap 95% intervals.

5.5. RQ5: Are conclusions robust to DQN tuning and task expansion?

The development-only sweep selects the vanilla 128-unit DQN with learning rate 3×10^{-4} , replay capacity 100,000, warm-up 512, and target interval 500. The Double-DQN candidate is not selected. In independent validation, the selected configuration has a better descriptive mean environment rank than the original comparator (1.25 versus 1.75), but no per-environment planned contrast survives Holm correction. FrozenLake 4x4 and 8x8 have positive uncorrected bootstrap intervals, while Taxi includes a large negative tail. Accordingly, the comparator is more extensively validated than the original choice, but the study does not claim a statistically established universal DQN improvement. A stricter acceptance criterion would require full 30-seed dueling-DQN, Double-DQN, A2C or PPO, and preferably convolutional MiniGrid comparators; because those full runs are not in the deposited evidence, all hybrid-versus-neural statements are phrased relative to the validated DQN used here.

Expanded MiniGrid evidence is heterogeneous (Fig. 4, Table 5). Count gating improves over the validated DQN after Holm correction only on Empty 8x8 (difference 0.596, interval [0.442, 0.750], $p = 0.000761$). The validated DQN has zero AUC on several layouts. Tabular Q-learning has the best descriptive mean environment rank (1.58) and substantially exceeds count gating on both DoorKey tasks and Empty 8x8 in raw means, although not every comparison survives the configured family correction. MiniGrid Four-

Table 4: Independent validation of the development-selected DQN. Positive differences favor the validated configuration.

Environment	Pairs	Mean difference [bootstrap 95% CI]	t-Holm p	Wins/losses/ties
CliffWalking	30	0.000 [0.000, 0.000]	1	0/0/30
FourRooms 11 slip	30	0.053 [-0.028, 0.136]	0.91	16/14/0
FourRooms 7	30	0.038 [-0.009, 0.086]	0.667	18/12/0
FourRooms 9 slip	30	0.009 [-0.054, 0.072]	1	16/14/0
FrozenLake 4x4	30	0.071 [0.011, 0.131]	0.232	20/9/1
FrozenLake 8x8	30	0.011 [0.002, 0.021]	0.232	16/9/5
MiniGrid Empty 5x5	10	0.000 [0.000, 0.000]	1	0/0/10
Taxi	30	-16.273 [-35.978, 1.003]	0.626	1/6/23

Rooms shows a small abstention-over-count difference of 0.012, with all ten pairs favoring abstention. These results broaden the failure evidence but do not establish a single best method.

5.6. RQ6: What does the application-oriented support shift show?

The application study completed all 180 planned runs and passed the result audit. Figure 5 and Table 6 show the central trade-off. Evaluation states are unsupported on approximately 39.2% of decision steps for every method. The fuzzy gate improves AUC over the validated DQN by 0.575 ([0.152, 0.987], t -Holm $p = 0.0256$, Wilcoxon-Holm $p = 0.0163$), but its 0.211 difference from count gating is not significant ([−0.171, 0.626]). It remains worse than support abstention by 2.520 and tabular Q-learning by 2.299; 29 of 30 paired seeds favor each comparator.

The outcome metrics explain why a single performance number is insufficient. Tabular Q-learning has the highest final success rate (0.751), while support abstention has lower success (0.544) but only 0.007 collision rate because its unsupported-state decision is the hold action. Fuzzy arbitration reaches 0.498 success but has 0.244 collision rate. Risk-zone occupancy is lower than collision exposure for all methods, but it follows the same cautionary message: return, success, collision, risk, and abstention must be read jointly. These results support a decision-support interpretation: a safe fallback can reduce harmful motion without creating transfer, whereas neural fallback can improve some learning comparisons while retaining deployment risk.

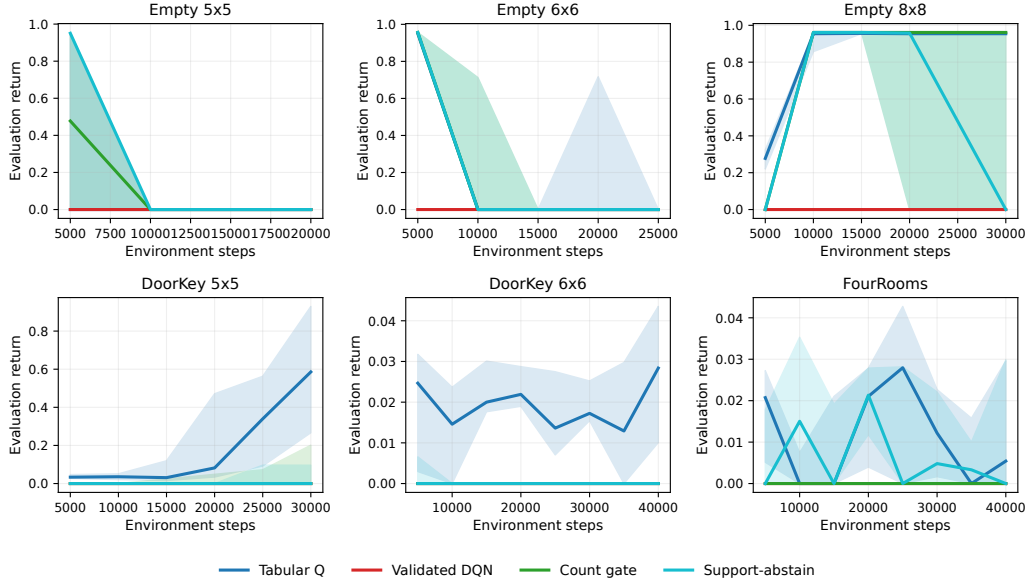


Figure 4: Median learning curves and interquartile bands for the six fully observable MiniGrid diagnostics, ten seeds per method.

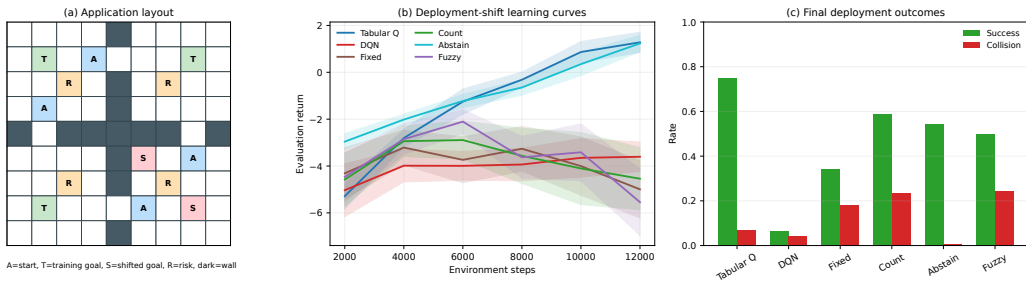


Figure 5: Application-inspired navigation under deployment goal shift. Panel (a) shows the controlled map: starts, recurring training goals, shifted deployment goals, risk cells, and walls. Panels (b) and (c) report learning curves and final deployment outcomes.

Table 5: Return AUC in the extended MiniGrid diagnostic. This family is post hoc and uses ten seeds; it is not pooled with compact confirmation.

Environment	Method	Seeds	Mean AUC [bootstrap 95% CI]	Median AUC
DoorKey 5x5	Count gate	10	0.041 [0.001, 0.083]	0
DoorKey 5x5	Support-abstain	10	0.033 [0.000, 0.080]	0
DoorKey 5x5	Tabular Q	10	0.210 [0.116, 0.309]	0.147
DoorKey 5x5	Validated DQN	10	0.000 [0.000, 0.000]	0
DoorKey 6x6	Count gate	10	0.000 [0.000, 0.000]	0
DoorKey 6x6	Support-abstain	10	0.000 [0.000, 0.001]	0
DoorKey 6x6	Tabular Q	10	0.021 [0.017, 0.026]	0.0189
DoorKey 6x6	Validated DQN	10	0.000 [0.000, 0.000]	0
Empty 5x5	Count gate	10	0.159 [0.064, 0.254]	0.159
Empty 5x5	Support-abstain	10	0.175 [0.094, 0.270]	0.159
Empty 5x5	Tabular Q	10	0.190 [0.079, 0.317]	0.158
Empty 5x5	Validated DQN	10	0.000 [0.000, 0.000]	0
Empty 6x6	Count gate	10	0.203 [0.107, 0.310]	0.12
Empty 6x6	Support-abstain	10	0.262 [0.167, 0.381]	0.12
Empty 6x6	Tabular Q	10	0.214 [0.143, 0.298]	0.12
Empty 6x6	Validated DQN	10	0.000 [0.000, 0.000]	0
Empty 8x8	Count gate	10	0.596 [0.442, 0.750]	0.577
Empty 8x8	Support-abstain	10	0.586 [0.452, 0.720]	0.577
Empty 8x8	Tabular Q	10	0.779 [0.652, 0.876]	0.865
Empty 8x8	Validated DQN	10	0.000 [0.000, 0.000]	0
FourRooms	Count gate	10	0.002 [0.000, 0.003]	0
FourRooms	Support-abstain	10	0.014 [0.012, 0.016]	0.0133
FourRooms	Tabular Q	10	0.017 [0.012, 0.021]	0.02
FourRooms	Validated DQN	10	0.002 [0.000, 0.003]	0

Table 6: Application-navigation outcomes at deployment shift. Rates are final-checkpoint seed means; AUC intervals are seed-bootstrap 95% intervals.

Method	Return AUC [95% bootstrap CI]	Success	Failure	Collision	Risk	Unsupported	Abstention
Count gate	-3.612 [-4.201, -3.043]	0.590	0.410	0.233	0.021	0.392	0.000
Fixed mixture	-3.773 [-4.490, -3.113]	0.341	0.659	0.183	0.016	0.393	0.000
Fuzzy adaptive	-3.401 [-3.861, -2.978]	0.498	0.502	0.244	0.017	0.393	0.000
Support abstention	-0.881 [-1.108, -0.649]	0.544	0.456	0.007	0.033	0.392	0.392
Tabular Q	-1.102 [-1.326, -0.891]	0.751	0.249	0.070	0.036	0.392	0.392
Validated DQN	-3.976 [-4.345, -3.630]	0.064	0.936	0.044	0.047	0.393	0.000

5.7. RQ7: Does fuzzy arbitration generalize, and what does it cost?

Table 7 reports the independently seeded compact validation. Relative to DQN, fuzzy arbitration improves FrozenLake 4x4 AUC by 0.291 ([0.244, 0.338], t -Holm $p = 1.12 \times 10^{-11}$), with all 30 seeds favoring the fuzzy method. On CliffWalking the mean difference is 925 with a positive bootstrap interval, but heavy tails leave the mean-based Holm test nonsignificant; the Wilcoxon-Holm sensitivity test is $p = 1.45 \times 10^{-4}$. The fuzzy gate does not significantly beat count gating in any of the three tasks. Under complete FourRooms support shift, its mean alpha is zero, so it matches neural fallback and is worse than both tabular and support-abstention methods in all 30 paired seeds.

Figure 6 adds a direct decision-surface sensitivity analysis for the fuzzy gate. It varies τ_N , κ , membership shape, consequents, and the explicit zero-support rule without retraining an RL agent. The important boundary remains invariant: at $N(s) = 0$, the configured neural-fallback fuzzy gate assigns zero memory weight. Lower τ_N moves the gate toward memory after fewer visits, higher τ_N delays memory use, and κ changes how rapidly TD-error evidence is treated as neural uncertainty. This analysis justifies the fuzzy surface as an interpretable ablation target rather than a hidden constant.

The ten-seed descriptive cost family in Table 8 and Figure 7 shows that fuzzy logic is not free. In application navigation, mean decision times are 134 μ s for tabular Q-learning, 349 μ s for support abstention, 442 μ s for DQN, 573 μ s for count gating, and 713 μ s for fuzzy arbitration. FourRooms shows the same ordering between count and fuzzy gates (485 versus 655 μ s). Exact-memory lower-bound estimates are about 5.4 KiB in application navigation and 32.8 KiB for fuzzy arbitration in FourRooms; the DQN’s zero denotes no exact-state table, not zero neural-model memory. Timings are machine- and contention-dependent descriptive measurements, not hardware-independent complexity claims.

Figure 8 summarizes the task-specific ranks. It makes the central boundary visible: count gating is competitive on recurrent tasks with a weak neural comparator, abstention protects held-out support, and tabular learning remains difficult to beat in compact discrete domains.

Table 7: Return AUC in the 30-seed fuzzy adaptive-gate validation.

Environment	Method	Return AUC [95% bootstrap CI]
CliffWalking	Count gate	-491.575 [-598.365, -400.000]
CliffWalking	Fuzzy adaptive	-400.017 [-467.333, -346.083]
CliffWalking	Support abstention	-412.547 [-488.410, -354.271]
CliffWalking	Tabular Q	-474.701 [-558.106, -401.679]
CliffWalking	Validated DQN	-1325.000 [-2150.000, -706.250]
FourRooms 9 support shift	Count gate	-2.280 [-2.323, -2.240]
FourRooms 9 support shift	Fuzzy adaptive	-2.280 [-2.329, -2.231]
FourRooms 9 support shift	Support abstention	-1.084 [-1.108, -1.062]
FourRooms 9 support shift	Tabular Q	-1.085 [-1.119, -1.053]
FourRooms 9 support shift	Validated DQN	-2.279 [-2.324, -2.234]
FrozenLake 4x4	Count gate	0.528 [0.498, 0.558]
FrozenLake 4x4	Fuzzy adaptive	0.505 [0.472, 0.538]
FrozenLake 4x4	Support abstention	0.524 [0.495, 0.553]
FrozenLake 4x4	Tabular Q	0.450 [0.412, 0.487]
FrozenLake 4x4	Validated DQN	0.214 [0.179, 0.253]

Table 8: Support, branch use, exact-memory lower bound, and decision time. The DQN exact-memory fields are zero because neural parameters are excluded.

Environment	Method	Unsupported	Memory branch	Neural branch	Abstention	Stored states	Estimated KiB	Decision time (us)
Application navigation	Count gate	0.430	0.437	0.563	0.000	198.5	5.43	573.3
Application navigation	Fuzzy adaptive	0.430	0.432	0.568	0.000	196.8	5.38	713.1
Application navigation	Support abstention	0.430	0.439	0.131	0.430	197.6	5.40	348.5
Application navigation	Tabular Q	0.430	0.570	0.000	0.430	199.0	5.44	134.1
Application navigation	Validated DQN	0.430	0.000	1.000	0.000	0.0	0.00	442.5
FourRooms 9 support shift	Count gate	1.000	0.000	1.000	0.000	1401.7	32.85	485.0
FourRooms 9 support shift	Fuzzy adaptive	1.000	0.000	1.000	0.000	1401.1	32.84	653.1
FourRooms 9 support shift	Support abstention	1.000	0.000	0.000	1.000	1524.3	35.73	168.9
FourRooms 9 support shift	Tabular Q	1.000	0.000	0.000	1.000	1903.6	44.62	132.4
FourRooms 9 support shift	Validated DQN	1.000	0.000	1.000	0.000	0.0	0.00	416.8

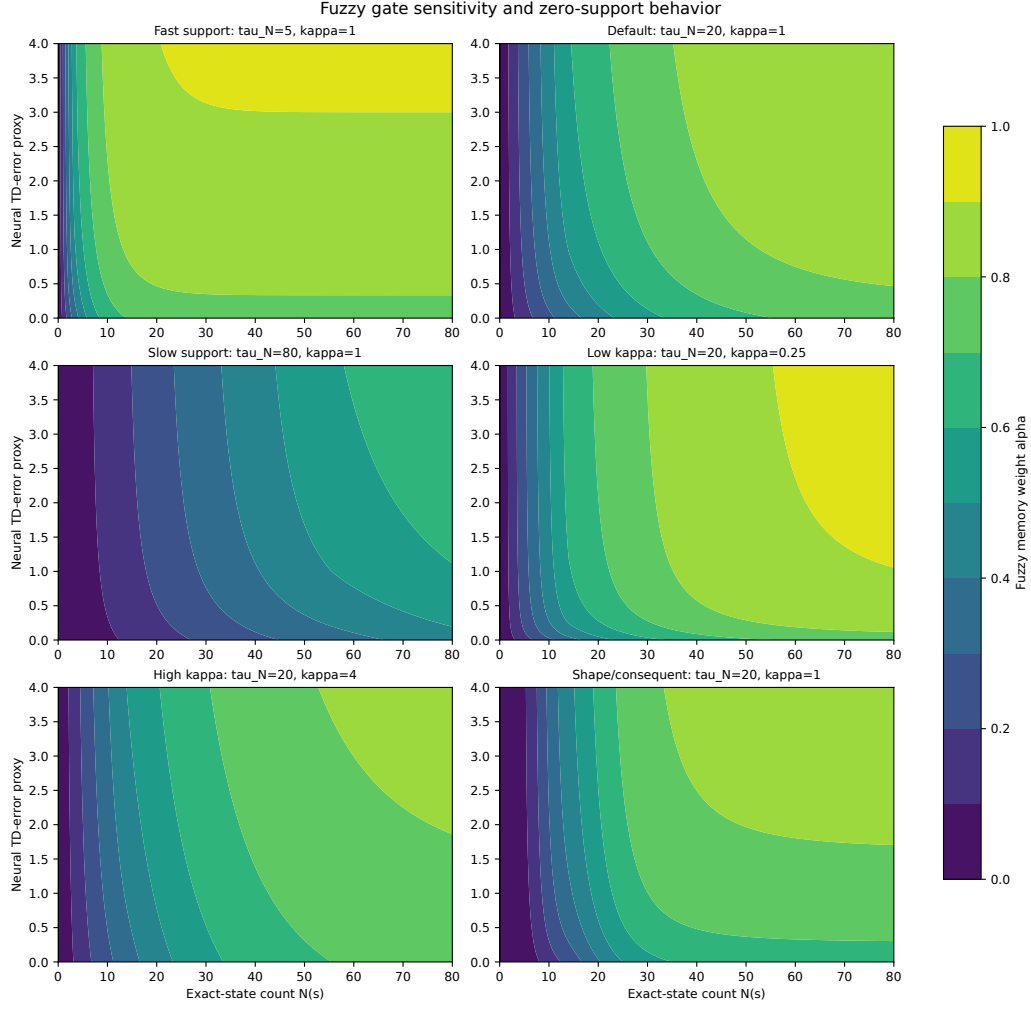


Figure 6: Analytic fuzzy-gate sensitivity to τ_N , κ , membership shape, consequents, and zero-support behavior. The vertical zero-count boundary shows the explicit neural-fallback setting used in the reported fuzzy baseline.

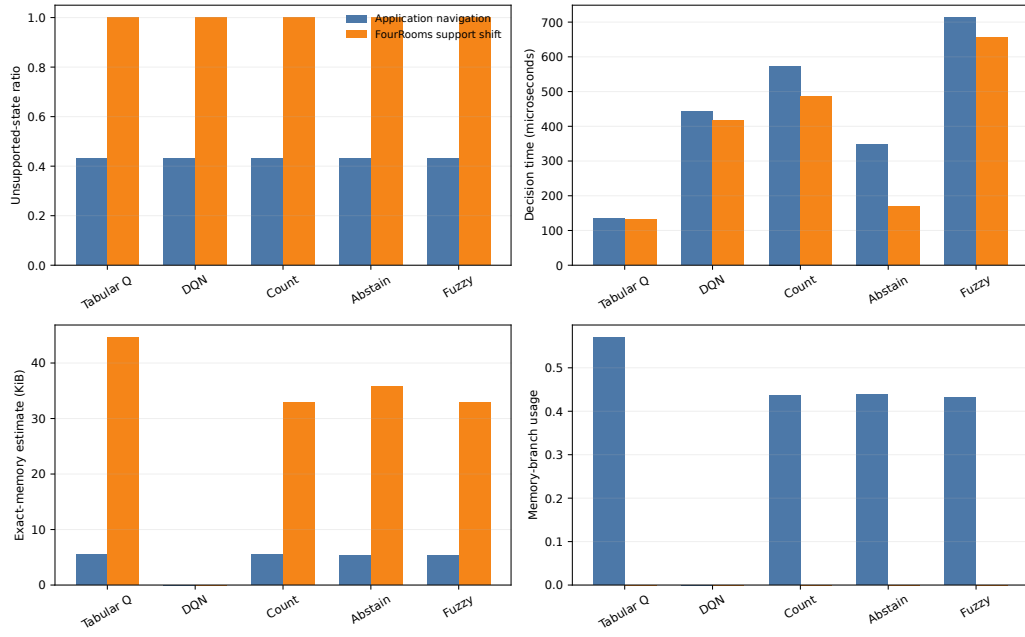


Figure 7: Descriptive cost and support metrics in the ten-seed analysis. Decision timers exclude environment stepping; memory is the documented exact-table lower bound.

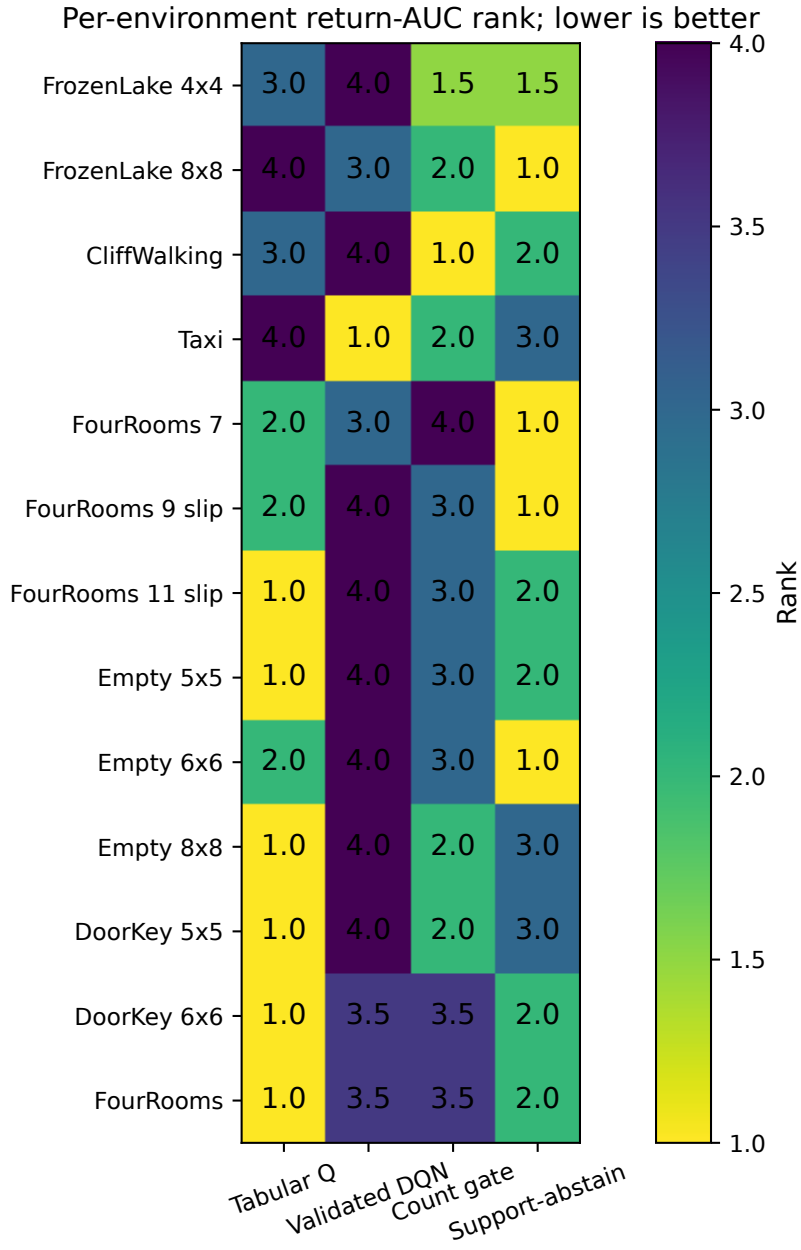


Figure 8: Per-environment return-AUC ranks across the compact expansion and extended MiniGrid diagnostic. Lower is better; ranks are descriptive because raw returns are not commensurate across tasks.

Auxiliary smoke validations for approximate support, fuzzy parameter variants, and dueling-DQN/A2C executability are included in the artifact rather than in the main results. They are useful for release auditing but are not performance evidence and should not be read as substitutes for full confirmatory runs.

6. Discussion

The evidence supports a narrow engineering interpretation. Count gating is a fallback for recurring exact states. When the neural estimator is weak and the table repeatedly observes the same keys, increasing tabular weight can improve learning relative to DQN. This occurs on both FrozenLake sizes and CliffWalking. It does not imply that the hybrid has learned a better general representation.

Exact-state memory is not a generalization mechanism. Held-out goals remove table support under the deliberately strict key definition, and the standard count gate responds by placing more weight on the neural estimator. The three-size replication shows that this design can be systematically harmful. Support abstention repairs the particular failure because it refuses unsupported neural preferences, but it does not create positive transfer and does not generally exceed tabular learning. This is also why the approximate-support comparator is separated from the exact-support claims: nearest-neighbor support may soften the boundary, but it must be evaluated as a different assumption, not retrofitted into the exact-key result.

Fuzzy arbitration provides a more graded mechanism but not a different support boundary. When counts recur, its support and TD-residual inputs can improve a weak DQN, as observed on FrozenLake 4x4 and in the application AUC comparison. It does not significantly improve over the simpler count gate. When support is exactly zero, the configured fuzzy baseline assigns zero memory weight and inherits the neural branch’s extrapolation behavior. Soft rules should therefore not be interpreted as calibrated confidence or a safety guarantee merely because they are continuous and interpretable.

The application case makes the engineering trade-off concrete. A hold fallback nearly removes collisions at unsupported states but sacrifices some final success relative to tabular learning. Because hold is deliberately safe in this controlled map, the collision reduction is a diagnostic result rather than a field-safety proof. Fuzzy and count gates continue to move, which can improve learning relative to DQN but can also raise collision exposure.

An operator-facing system could use the logged support signal to request evidence, hand control to a verified planner, or hold position. The present experiment measures that decision boundary; it does not validate any particular operational fallback.

Strong tabular baselines materially limit practical advantage in these small discrete tasks. Taxi is an exception in which count gating improves over tabular learning while the validated DQN remains competitive. MiniGrid is mixed: count gating helps relative to the tested DQN on Empty 8x8, but tabular learning has the best overall rank. The results should therefore be interpreted as a mechanism and evaluation study, not as evidence that exact-state memory should be added to arbitrary deep-RL systems.

DQN sensitivity remains a major caveat. The broader development sweep and independent validation are stronger than a single hand-selected baseline, yet the selected configuration does not significantly improve every task and fails on several MiniGrid layouts. More capable architectures, convolutional representations, longer budgets, and environment-specific tuning could change the magnitude of hybrid-versus-neural differences. The claim is explicitly about the tested, audited comparator family.

7. Limitations

The study uses compact simulated benchmarks, discrete observations or explicitly encoded discrete images, and limited MiniGrid scale. It includes no continuous-control task, pixel-based partial observability, real deployment, or safety-critical intervention. The MiniGrid network is an MLP over a fully observable flattened image rather than a convolutional architecture. The application environment is intentionally controlled and does not model sensor noise, moving obstacles, localization error, actuator dynamics, human hand-over, or physical consequences. Its collision and risk metrics must not be read as field-safety evidence.

The DQN comparator is sensitive to hyperparameters and compute budget. The development sweep is broader than the original selection but remains finite. The revised code now includes dueling-DQN and A2C agents plus full-run configuration templates for reviewer-requested follow-up studies. Because the deposited stronger-baseline results are intentionally small and run in the dependency-free fallback environment, they verify executability but do not support broad performance claims about stronger neural baselines. A future acceptance-oriented revision should execute those comparators, and

preferably PPO and convolutional MiniGrid models, at the same seed scale as the main families. The support-abstention idea was discovered after observing the original FourRooms failure; the new independent seeds support the repair, but the sequence must remain visible. The MiniGrid expansion is diagnostic and uses ten seeds per task. Some returns are heavy-tailed, particularly on CliffWalking, FrozenLake 8x8, and Taxi, so mean and rank-based conclusions can differ.

Exact-state tables grow with the number of encountered keys. The conclusions apply to exact-state support and should not be generalized to all memory-augmented RL, nearest-neighbor episodic control, uncertainty estimation, or abstention methods. The added approximate-support comparator replaces exact keys with kNN/kernel support and is covered by an audited smoke-validation run and a full-run configuration template; that limited deposited result confirms that nearby memory can be queried, but it is not a full-scale performance study. Computational limits prevented investigation of Atari-scale or continuous-control domains. These remain future tests rather than implied evidence. The TD-residual uncertainty proxy is uncalibrated, and the fuzzy rule consequents are fixed rather than learned. Decision timing was measured on one CPU while independent runs were parallelized; it supports within-artifact descriptive comparison but not universal latency ranking. The memory estimate excludes Python-object overhead and neural parameters.

8. Reproducibility and declarations

The artifact contains source code, all JSON configurations, raw seed-level results, generated summaries, heavy-tail and cross-environment diagnostics, figures, tests, package versions, commit identifiers, and integrity audits. The complete reported revision families contain 4,145 independent method–environment–seed runs, including 730 new full runs for the application, adaptive-gate, and cost/support analyses. The package additionally contains 38 audited dependency-free smoke runs for approximate support, fuzzy ablation, and stronger neural comparator code paths. Quick reanalysis runs with `python scripts/reproduce_all.py -quick`; full retraining uses `python scripts/reproduce_all.py -full`.

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Ethics. The study uses simulated environments and no human participants, animals, personal data, or identifiable records.

CRedit authorship contribution statement. Ercan Erkalkan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing—original draft, Writing—review and editing.

Data and code availability. The maintained source and the ASOC revision artifact are available at <https://github.com/ErcanErkalkan/confidence-gated-q>. The persistent artifact concept DOI is <https://doi.org/10.5281/zenodo.20578927> [27]. If a journal-specific Zenodo release is minted, the version-specific DOI for that release should replace the concept DOI in the final proofs. The supplementary artifact submitted with this manuscript contains the exact code, configurations, seed-level results, tests, generated assets, and audit records used to produce the reported claims.

9. Conclusion

Simple tabular–neural gating is useful only under a restricted condition: exact states must recur often enough for the table to provide evidence. Under that condition, count gating improves the tested validated DQN on selected compact tasks. Under held-out support shift, the same gate can be harmful because zero support delegates to neural extrapolation. Independent replication shows that support abstention repairs this specific FourRooms failure, but does not become a universal solution or generally outperform tabular Q-learning. Fuzzy support- and uncertainty-aware arbitration improves selected DQN comparisons, including the controlled navigation AUC, but does not generally beat count gating and fails with the same neural fallback under zero support. Its additional decision-time cost is measurable. The main contribution is therefore a reproducible pre-deployment diagnostic and a clearly measured boundary: exact memory helps under recurrence, adaptive arbitration helps only when reliability signals are informative, and explicit fallback remains important under support shift. Future work should test calibrated uncertainty, learned fuzzy rules, verified fallback controllers, larger domains, and continuous control without weakening the separation between development, confirmation, diagnosis, and replication.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used OpenAI Codex for language editing, code review, and artifact organization. After using this service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

- [1] C. J. C. H. Watkins, P. Dayan, Q-learning, *Machine Learning* 8 (3–4) (1992) 279–292. doi:10.1007/BF00992698.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, et al., Human-level control through deep reinforcement learning, *Nature* 518 (2015) 529–533. doi:10.1038/nature14236.
- [3] H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30, 2016. doi:10.1609/aaai.v30i1.10295.
- [4] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, 2018, pp. 3207–3214. doi:10.1609/aaai.v32i1.11694.
- [5] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, M. G. Bellemare, Deep reinforcement learning at the edge of the statistical precipice, in: *Advances in Neural Information Processing Systems*, Vol. 34, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- [6] L. A. Zadeh, Soft computing and fuzzy logic, *IEEE Software* 11 (6) (1994) 48–56. doi:10.1109/52.329401.
- [7] C. Blundell, B. Uria, A. Pritzel, et al., Model-free episodic control, in: *Proceedings of the 33rd International Conference on Machine Learning*, Vol. 48 of *Proceedings of Machine Learning Research*, 2016, pp. 1617–1626. URL <https://proceedings.mlr.press/v48/blundell116.html>

- [8] A. Pritzel, B. Uria, S. Srinivasan, et al., Neural episodic control, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, 2017, pp. 2827–2836.
URL <https://proceedings.mlr.press/v70/pritzel17a.html>
- [9] Z. Lin, T. Zhao, G. Yang, L. Zhang, Episodic memory deep q-networks, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, 2018, pp. 2433–2439. doi:10.24963/ijcai.2018/337.
- [10] S. Hansen, A. Pritzel, P. Sprechmann, et al., Fast deep reinforcement learning using online adjustments from the past, in: Advances in Neural Information Processing Systems, Vol. 31, 2018.
URL <https://proceedings.neurips.cc/paper/2018/hash/1de7d2b90d554be9f0db1c338e80197d-Abstract.html>
- [11] H. Hu, J. Ye, G. Zhu, Z. Ren, C. Zhang, Generalizable episodic memory for deep reinforcement learning, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 4380–4390.
URL <https://proceedings.mlr.press/v139/hu21d.html>
- [12] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, R. Munos, Unifying count-based exploration and intrinsic motivation, in: Advances in Neural Information Processing Systems, Vol. 29, 2016.
URL <https://proceedings.neurips.cc/paper/2016/hash/afda332245e2af431fb7b672a68b659d-Abstract.html>
- [13] I. Osband, C. Blundell, A. Pritzel, B. Van Roy, Deep exploration via bootstrapped dqn, in: Advances in Neural Information Processing Systems, Vol. 29, 2016.
URL <https://proceedings.neurips.cc/paper/2016/hash/8d8818c8e140c64c743113f563cf750f-Abstract.html>
- [14] N. D. Daw, Y. Niv, P. Dayan, Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control, *Nature Neuroscience* 8 (2005) 1704–1711. doi:10.1038/nn1560.

- [15] P. Qin, T. Zhao, Knowledge guided fuzzy deep reinforcement learning, *Expert Systems with Applications* 264 (2025) 125823. doi:10.1016/j.eswa.2024.125823.
- [16] F. B. Tilahun, Fuzzy-based predictive deep reinforcement learning for robust and constrained optimal control of industrial solar thermal plants, *Applied Soft Computing* 159 (2024) 111432. doi:10.1016/j.asoc.2024.111432.
- [17] L. Chen, C. Jiang, Y. Huang, W. Wu, Uncertainty-aware cloud resource management via fuzzy-adaptive hierarchical reinforcement learning, *Applied Soft Computing* 186 (2026) 114157. doi:10.1016/j.asoc.2025.114157.
- [18] G. Kahn, A. Villafior, V. Pong, P. Abbeel, S. Levine, Uncertainty-aware reinforcement learning for collision avoidance, *arXiv preprint arXiv:1702.01182* (2017).
URL <https://arxiv.org/abs/1702.01182>
- [19] K. Cobbe, O. Klimov, C. Hesse, T. Kim, J. Schulman, Quantifying generalization in reinforcement learning, in: *Proceedings of the 36th International Conference on Machine Learning*, Vol. 97 of *Proceedings of Machine Learning Research*, 2019, pp. 1282–1289.
URL <https://proceedings.mlr.press/v97/cobbe19a.html>
- [20] R. Kirk, A. Zhang, E. Grefenstette, T. Rocktaschel, A survey of zero-shot generalisation in deep reinforcement learning, *Journal of Artificial Intelligence Research* 76 (2023) 201–264. doi:10.1613/jair.1.14174.
- [21] S. Fujimoto, D. Meger, D. Precup, Off-policy deep reinforcement learning without exploration, in: *Proceedings of the 36th International Conference on Machine Learning*, Vol. 97 of *Proceedings of Machine Learning Research*, 2019, pp. 2052–2062.
URL <https://proceedings.mlr.press/v97/fujimoto19a.html>
- [22] A. Kumar, A. Zhou, G. Tucker, S. Levine, Conservative q-learning for offline reinforcement learning, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 1179–1191.
URL <https://proceedings.neurips.cc/paper/2020/hash/0d2b2061826a5df3221116a5085a6052-Abstract.html>

- [23] Y. Geifman, R. El-Yaniv, SelectiveNet: A deep neural network with an integrated reject option, in: Proceedings of the 36th International Conference on Machine Learning, Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 2151–2159.
URL <https://proceedings.mlr.press/v97/geifman19a.html>
- [24] J. Pineau, P. Vincent-Lamarre, K. Sinha, et al., Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program, Journal of Machine Learning Research 22 (164) (2021) 1–20.
URL <https://www.jmlr.org/papers/v22/20-303.html>
- [25] M. Chevalier-Boisvert, B. Dai, M. Towers, et al., Minigrid and mini-world: Modular and customizable reinforcement learning environments for goal-oriented tasks, arXiv preprint arXiv:2306.13831 (2023).
URL <https://arxiv.org/abs/2306.13831>
- [26] M. Towers, A. Kwiatkowski, J. Terry, et al., Gymnasium: A standard interface for reinforcement learning environments, arXiv preprint arXiv:2407.17032 (2024).
URL <https://arxiv.org/abs/2407.17032>
- [27] E. Erkalkan, Support-boundary diagnostic for tabular-neural reinforcement learning: Reproducibility artifact, Zenodo/GitHub, [software] Persistent concept DOI for the reproducibility artifact. Source repository: <https://github.com/ErcanErkalkan/confidence-gated-q> (2026). doi:10.5281/zenodo.20578927.
URL <https://doi.org/10.5281/zenodo.20578927>